

ITU - MASTER 1
PROJET MOBILE**TATITRA****Application citoyenne de signalement des problèmes d'infrastructures publiques**

Cahier des charges fonctionnel et technique

Principe du projet

Permettre à un citoyen de créer un signalement géolocalisé avec photo, même sans connexion Internet, puis de synchroniser automatiquement les données avec une plateforme destinée à l'administration.

Élément	Valeur
Type de projet	Application mobile Android + API + interface Web d'administration
Équipe	Binôme
Durée cible	7 jours
Approche	MVP / Offline-first
Public cible	Citoyens et agents administratifs
Version du document	1.2 - Septembre 2026

Document de conception académique. Les données de démonstration décrites dans ce document peuvent être simulées et ne doivent pas être présentées comme des incidents réels.

Sommaire

1. Résumé exécutif
2. Contexte, justification et problématique
3. Objectifs et périmètre
4. Parties prenantes et rôles
5. Besoins fonctionnels
6. Parcours utilisateurs et UI
7. Exigences offline-first
8. Architecture technique
9. Données manipulées et stockées
10. Données de test et stratégie de démonstration
11. API REST
12. Interface Web d'administration
13. Sécurité, confidentialité et permissions
14. Exigences non fonctionnelles
15. Modèle de données
16. Extraits de code proposés
17. Tests et critères d'acceptation
18. Organisation du binôme et planning 7 jours
19. Risques et stratégie de réduction
20. Déploiement et environnement
21. Fonctionnalités futures
22. Livrables
23. Références et sources
24. Annexes

1. Résumé exécutif

TATITRA est une application mobile citoyenne destinée à faciliter la remontée d'informations concernant les dégradations et anomalies observées sur les infrastructures et espaces publics : routes endommagées, nids-de-poule, éclairage public défectueux, canaux bouchés, dépôts de déchets, ponts ou ouvrages détériorés, problèmes de drainage et autres incidents similaires.

Le citoyen crée un signalement à partir de son smartphone en indiquant une catégorie, une description, une photo et une position GPS. L'application est conçue en mode offline-first : le signalement est d'abord sauvegardé localement puis synchronisé vers le serveur dès qu'une connexion Internet est disponible.

L'administration n'utilise pas une seconde application mobile. Elle dispose d'une interface Web légère permettant de consulter les signalements, voir leurs détails et modifier leur statut. La résolution est bidirectionnelle : le citoyen comme l'administrateur peut signaler qu'un problème semble résolu ; l'autre partie est alors invitée à confirmer. Si aucune confirmation croisée n'est obtenue dans un délai de sept jours, le signalement est automatiquement rouvert comme non résolu / endommagé. Cette architecture limite la charge de développement tout en représentant fidèlement un système de terrain réaliste.

Décision de périmètre

Dans la version d'examen, le cœur à réussir est : photo + GPS + Room offline + synchronisation + API + base distante + interface admin + validation croisée de résolution + réouverture automatique après 7 jours sans confirmation. La carte globale, l'IA, les statistiques avancées et les notifications push système sont des bonus.

2. Contexte, justification et problématique

2.1 Contexte

Les services publics doivent gérer un territoire étendu et de nombreux équipements. Une application de signalement peut compléter les mécanismes existants en permettant aux citoyens de transmettre des informations structurées et géolocalisées. Le Ministère des Travaux Publics présente l'entretien et la modernisation du réseau routier comme des éléments importants du développement et souligne aussi l'implication des citoyens dans les défis d'infrastructure.

TATITRA ne prétend pas remplacer les procédures officielles ni déterminer automatiquement la responsabilité d'un ministère, d'une commune ou d'un autre service. Le prototype démontre un canal numérique de remontée, de centralisation et de suivi.

2.2 Problématique

Problématique centrale

Comment permettre aux citoyens de signaler rapidement un problème d'infrastructure publique, y compris lorsque la connexion Internet est absente ou instable, tout en offrant à l'administration un moyen simple de centraliser et suivre les signalements ?

2.3 Valeur attendue

- Réduire la friction pour transmettre une information de terrain.

- Structurer les signalements avec photo, GPS, date, catégorie et description.
- Conserver un signalement lorsque le réseau est indisponible.
- Fournir une vue centralisée aux agents administratifs.
- Créer une base exploitable à terme pour le suivi et la priorisation des interventions.

3. Objectifs et périmètre

3.1 Objectif principal

Développer un MVP Android permettant la création, la conservation hors ligne et la synchronisation de signalements citoyens géolocalisés, accompagné d'une API REST et d'une interface Web d'administration.

3.2 Objectifs spécifiques

- Créer un signalement en moins de quelques étapes.
- Capturer ou sélectionner une photo de démonstration.
- Récupérer la position GPS ou permettre une saisie/confirmation de localisation.
- Enregistrer localement les données avec Room.
- Identifier clairement les signalements non synchronisés.
- Synchroniser via WorkManager dès que le réseau est disponible.
- Envoyer les données vers une API REST sécurisée.
- Stocker les données structurées dans PostgreSQL/Supabase et les images dans un stockage objet.
- Permettre à un administrateur de consulter et mettre à jour le statut.
- Permettre au citoyen ou à l'administrateur de proposer qu'un signalement est résolu et d'en informer l'autre partie.
- Exiger une confirmation croisée de la résolution et rouvrir automatiquement le signalement après 7 jours sans confirmation.

3.3 MVP obligatoire et bonus

Fonctionnalité	MVP	Commentaire
Création de signalement	Oui	Fonction principale
Catégorie + description	Oui	Champs obligatoires
Photo	Oui	Caméra ou galerie
GPS	Oui	Coordonnées réelles du téléphone si autorisées
Room local	Oui	Offline-first
Liste « Mes signalements »	Oui	Affiche statut local/serveur
WorkManager	Oui	Synchronisation différée
API REST	Oui	Backend commun mobile/admin
PostgreSQL / Supabase	Oui	Données distantes
Admin Web	Oui	Liste, détail, changement de statut
Validation croisée de résolution	Oui	Citoyen ou admin propose ; l'autre confirme

Fonctionnalité	MVP	Commentaire
Réouverture automatique à J+7	Oui	Sans confirmation croisée : retour NON_RESOLU
Notifications in-app	Oui	Événement visible dans l'app/admin ; push système en bonus
Carte globale	Bonus	À ajouter seulement si le MVP est stable
Push Firebase	Bonus	Non indispensable
Authentification complète	Bonus	Peut être simulée ou simplifiée
IA reconnaissance image	Non	Perspective future
Vote/communauté	Non	Hors périmètre examen

4. Parties prenantes et rôles

Acteur	Description	Actions principales
Citoyen	Utilisateur de l'application Android	Créer, enregistrer, synchroniser et consulter ses signalements ; proposer ou confirmer une résolution ; signaler qu'un problème reste endommagé.
Administrateur	Agent simulé d'un service public	Consulter les signalements, ouvrir un détail, changer le statut, proposer ou confirmer une résolution et recevoir les notifications citoyennes.
Système Android	Composants du terminal	Caméra, GPS, stockage local, détection réseau, tâches WorkManager.
Backend	Service central	Validation, stockage, exposition des API, gestion des statuts.
Stockage objet	Service photos	Stocker les fichiers image et retourner une URL.

5. Besoins fonctionnels

5.1 Fonctions côté citoyen

ID	Exigence	Priorité
F-CIT-01	Afficher l'accueil et un accès rapide à « Nouveau signalement ».	Haute
F-CIT-02	Choisir une catégorie de problème.	Haute
F-CIT-03	Saisir une description.	Haute

ID	Exigence	Priorité
F-CIT-04	Prendre une photo ou choisir une image de la galerie pour une démonstration.	Haute
F-CIT-05	Récupérer latitude et longitude via le GPS.	Haute
F-CIT-06	Enregistrer immédiatement le signalement en local.	Haute
F-CIT-07	Afficher l'état de synchronisation.	Haute
F-CIT-08	Synchroniser automatiquement lorsque le réseau revient.	Haute
F-CIT-09	Afficher la liste des signalements locaux et synchronisés.	Haute
F-CIT-10	Consulter le détail et le statut.	Haute
F-CIT-11	Proposer « problème résolu » lorsque le citoyen constate la réparation.	Haute
F-CIT-12	Confirmer une résolution proposée par l'administration.	Haute
F-CIT-13	Indiquer « Toujours endommagé » pour rouvrir immédiatement un signalement.	Haute
F-CIT-14	Voir une notification in-app lorsqu'un administrateur propose une résolution ou modifie le statut.	Haute
F-CIT-15	Afficher une carte des signalements.	Bonus
F-CIT-16	Recevoir une notification push système.	Bonus

5.2 Fonctions côté administrateur

ID	Exigence	Priorité
F-ADM-01	Ouvrir un tableau de bord Web.	Haute
F-ADM-02	Lister les signalements reçus.	Haute
F-ADM-03	Afficher photo, catégorie, description, GPS, date et statut.	Haute
F-ADM-04	Modifier les statuts de traitement : ENVOYE, A_VERIFIER, PRIS_EN_CHARGE, REJETE.	Haute
F-ADM-05	Proposer « problème résolu » et notifier le citoyen.	Haute
F-ADM-06	Confirmer une résolution proposée par le citoyen ou la refuser si le traitement n'est pas terminé.	Haute
F-ADM-07	Voir les signalements	Haute

ID	Exigence	Priorité
	automatiquement rouverts après expiration du délai de 7 jours.	
F-ADM-08	Filtrer par statut ou catégorie.	Moyenne
F-ADM-09	Ajouter un commentaire interne.	Bonus
F-ADM-10	Afficher des statistiques.	Bonus

5.3 Résolution, confirmation croisée et réouverture

Pour éviter qu'un signalement soit fermé trop tôt ou reste bloqué parce qu'une partie a oublié de notifier l'autre, TATITRA utilise une validation croisée. Le citoyen et l'administrateur disposent tous les deux d'une action « Signaler comme résolu ». Cette action ne ferme pas immédiatement le dossier : elle crée une proposition de résolution et notifie l'autre partie.

```

PRIS_EN_CHARGE / NON_RESOLU
|
+-- Admin propose RESOLU -----> RESOLUTION_A_CONFIRMER
|
|
|
|
|
|
|
|
+-- Citoyen propose RESOLU -----> RESOLUTION_A_CONFIRMER
|
|
|
|
|
|
|
|
+-- Admin confirme -----> RESOLU_CONFIRME
+-- Admin refuse -----> REOUVERT_NON_RESOLU
+-- Pas de reponse 7 j -> REOUVERT_NON_RESOLU

```

Règle des 7 jours

Une résolution proposée reste provisoire tant que l'autre partie ne l'a pas confirmée. La date limite est fixée à J+7. Sans confirmation croisée avant cette date, le système remet automatiquement le signalement en état REOUVERT_NON_RESOLU / ENDOMMAGE afin qu'il réapparaisse dans les dossiers à traiter.

La réouverture automatique est exécutée côté serveur afin de fonctionner même si le téléphone du citoyen reste hors ligne. Pour le MVP, un job planifié Node.js (par exemple node-cron) peut vérifier périodiquement les résolutions arrivées à échéance.

6. Parcours utilisateurs et UI

6.1 Navigation mobile proposée

```

Accueil
|
+-- Nouveau signalement
|   +-- Catégorie
|   +-- Photo / Galerie
|   +-- GPS
|   +-- Description
|   +-- Enregistrer
|

```

```

+-- Mes signalements
    +-- Détail
    +-- Statut / Synchronisation
    +-- Signaler comme résolu
    +-- Confirmer résolu / Toujours endommagé

```

6.2 Écran Accueil

```

TATITRA
Bonjour

[ + NOUVEAU SIGNALEMENT ]

Mes signalements
- 2 en attente de synchronisation
- 5 envoyés
- 1 résolution à confirmer
- 3 résolus

[Accueil] [Signalements] [Profil]

```

6.3 Écran Nouveau signalement

```

NOUVEAU SIGNALEMENT

Catégorie      [ ROUTE v ]
Photo          [ Prendre ] [ Galerie ]
Localisation   GPS détecté
Latitude       -18.xxxxxx
Longitude      47.xxxxxx
Description    [.....]

[ ENREGISTRER ]

```

6.4 Écran Mes signalements

```

MES SIGNALEMENTS

ROUTE - Nid-de-poule
En attente de synchronisation

DECHETS - Dépôt sauvage
Envoyé

ECLAIRAGE - Lampadaire
Résolu

```

6.5 Confirmation de résolution

```

SIGNALEMENT #125
Route endommagée

Statut : RESOLUTION A CONFIRMER

```

Proposée par : Administration
 Délai restant : 5 jours

[CONFIRMER : C EST RESOLU]
 [TOUJOURS ENDOMMAGE]

Si le citoyen constate lui-même la réparation :
 [SIGNALER COMME RESOLU]

L'interface Web admin présente les mêmes actions lorsque la proposition vient du citoyen : confirmer la résolution ou indiquer que le traitement n'est pas terminé.

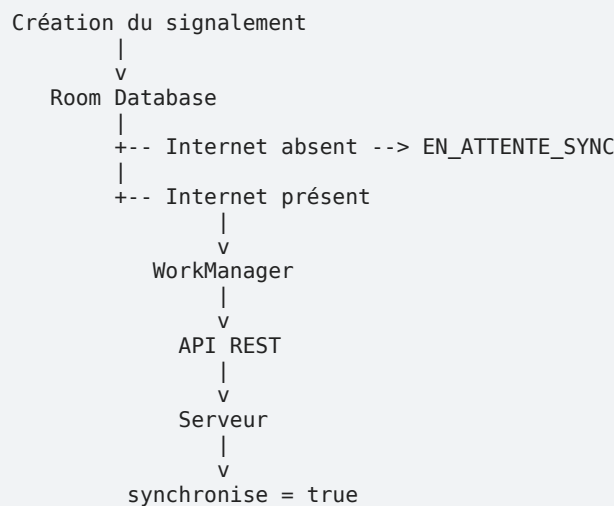
6.6 Principes UI/UX

- Interface simple et lisible, peu d'étapes.
- Bouton principal très visible pour créer un signalement.
- Statuts différenciés par texte et icône, pas uniquement par couleur.
- Messages explicites en cas de mode hors ligne.
- Possibilité d'utiliser le français dans le MVP et de prévoir Malagasy/Français comme évolution.
- Pas de dépendance obligatoire à une carte pour créer un signalement.

7. Exigences offline-first

7.1 Principe

Chaque signalement est d'abord sauvegardé localement. Le réseau n'est donc pas une condition préalable à la création. La synchronisation est un second processus.



7.2 Données locales minimales

Champ	Type	Rôle
idLocal	UUID/String	Identifiant généré sur le téléphone
categorie	String	Type de problème

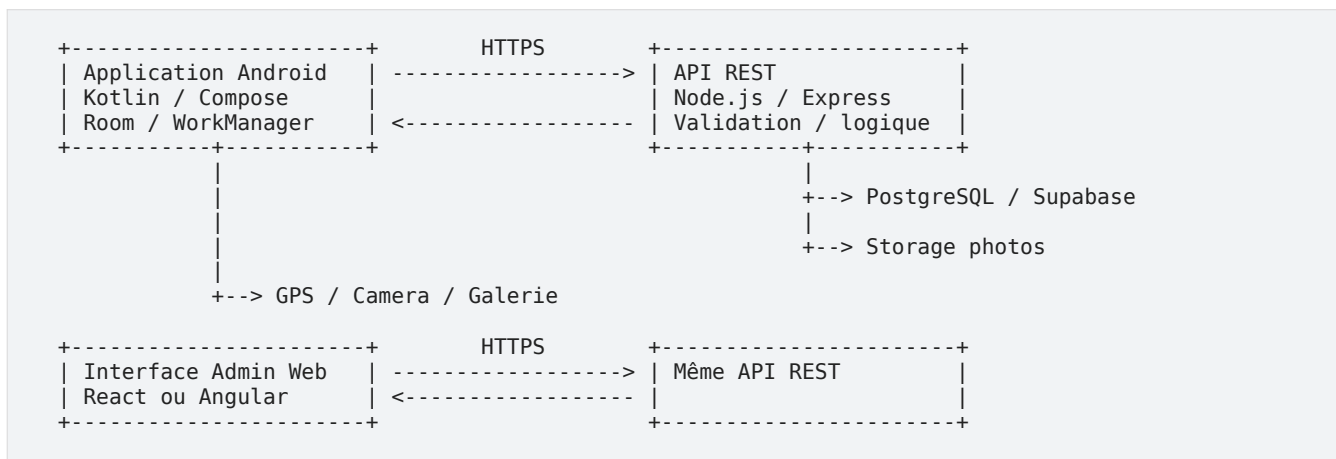
Champ	Type	Rôle
description	String	Description libre
latitude	Double	Position GPS
longitude	Double	Position GPS
photoPath	String?	Chemin local de l'image
statut	String	État métier connu localement
synchronise	Boolean	false avant succès serveur
serverId	String?	ID renvoyé par le backend
dateCreation	Long/DateTime	Date de création

7.3 Gestion des échecs

- En cas d'erreur réseau, conserver `synchronise=false`.
- Ne pas supprimer la photo locale avant confirmation de l'upload.
- Prévoir une stratégie de retry avec WorkManager.
- Éviter la duplication serveur grâce à un identifiant client unique ou une clé d'idempotence.
- Afficher un bouton « Réessayer » en plus de la synchronisation automatique si nécessaire.

8. Architecture technique

8.1 Architecture globale



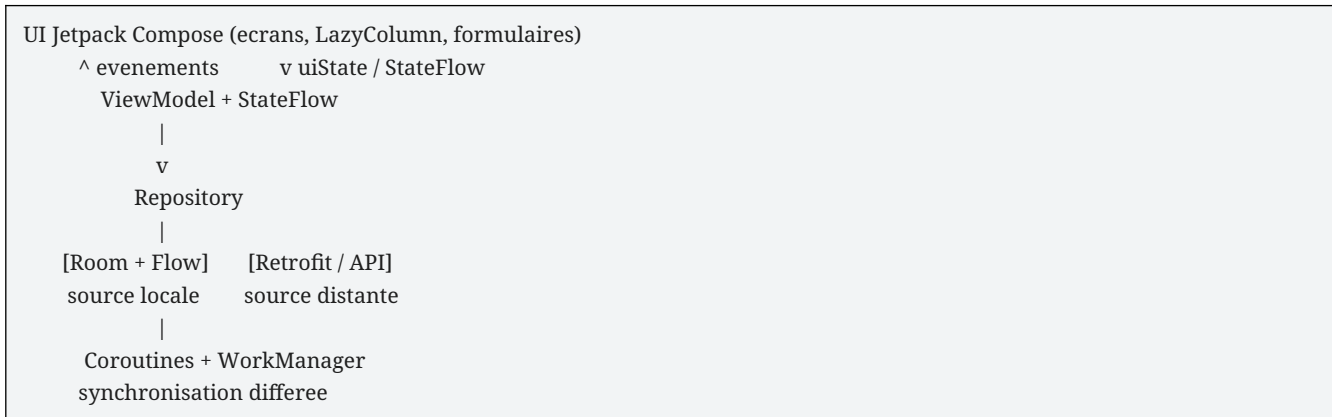
8.2 Stack recommandée

Couche	Technologie	Justification
Mobile	Kotlin + Jetpack Compose + Material 3	UI déclarative moderne, cohérente avec le module M1.
Architecture Android	MVVM + ViewModel + StateFlow	État durable, flux unidirectionnel et séparation UI / logique.

Couche	Technologie	Justification
Stockage local	Room + Flow	Source de vérité locale : persistance et observation réactive.
Synchronisation	WorkManager	Reprise différée sous contrainte réseau, y compris après fermeture.
HTTP	Retrofit + OkHttp + Coroutines	Appels REST asynchrones avec fonctions suspend.
Photo	CameraX + sélecteur galerie	Capture ou démonstration à domicile.
GPS	Fused Location Provider / Location Services	Coordonnées du terminal.
Backend	Node.js + Express	Développement rapide dans un délai de 7 jours.
Base distante	PostgreSQL / Supabase	Relationnel, pratique pour statuts et filtres.
Images	Supabase Storage	Stockage de fichiers séparé de la base.
Admin Web	React ou Angular	Une page de gestion suffit au MVP.
Versioning	Git + GitHub	Travail en binôme et traçabilité.

8.3 Architecture Android détaillée - flux unidirectionnel

TATITRA adopte l'architecture enseignée dans le module M1 : l'interface Compose observe un état produit par un ViewModel, le ViewModel délègue l'accès aux données à un Repository, et le Repository arbitre entre la base locale Room et l'API distante. Room reste la source de vérité locale ; le réseau alimente et synchronise cette base lorsqu'il est disponible.



- L'UI Compose affiche l'état et remonte les événements ; elle ne dialogue pas directement avec Room ou Retrofit.
- Le ViewModel conserve l'état utile lors des créations d'Activity et expose un StateFlow en lecture seule.
- Le Repository constitue le point d'accès unique aux données et décide quand lire localement ou synchroniser à distance.
- Les opérations disque et réseau sont réalisées avec des coroutines et des fonctions suspend afin de ne jamais bloquer le thread de l'interface.
- WorkManager assure la reprise de la synchronisation sous contrainte réseau et survit à la fermeture de l'application.
- remember est réservé aux états purement visuels et temporaires ; les données métier restent dans ViewModel / Room.

8.4 Techniques du module M1 mobilisées dans TATITRA

Les choix ci-dessous reprennent les concepts étudiés dans les documents pédagogiques fournis et les appliquent directement aux besoins de TATITRA. Les éléments marqués « optionnel » ne doivent pas retarder le MVP.

Technique / concept	Application dans TATITRA	Document du module	MVP
Kotlin : null safety + data classes	Modèles Signalement/Utilisateur ; champs GPS, photo ou serverId explicitement nullable ; éviter !!.	Séance 1 / Mini-TP Kotlin	Oui
Coroutines : suspend / launch	Room, Retrofit, upload et synchronisation sans bloquer le thread UI.	Séance 2 - Coroutines	Oui
Cycle de vie + Logcat	Tester rotation/recréation, suivre les workers et diagnostiquer les erreurs Android.	Séance 3 - Anatomie Android	Oui
Jetpack Compose + état	Écrans déclaratifs, Material 3, formulaires et LazyColumn pour les listes.	Séance 4 - Compose	Oui
Navigation Compose	NavHost, routes, argument signalementId et retour via back stack.	Séance 5 - Navigation	Oui
MVVM + StateFlow	ViewModel comme détenteur de l'état ; UI observe, événements remontent.	Séance 6 - MVVM	Oui
Repository	Isole la provenance des données et arbitre Room / API.	Séance 6 + état de l'art	Oui
Room + Flow	Base locale source de vérité ; listes et statuts réactifs, persistance hors ligne.	Séance 7 - Room	Oui
DataStore	Langue, onboarding terminé, date de dernière synchronisation et préférences simples.	Séance 7 - Room/DataStore	Optionnel utile
Retrofit + WorkManager	API distante et reprise automatique des synchronisations lorsque le réseau revient.	Séance 7 - projection vraie app	Oui
Intent implicite	Partage d'un identifiant ou résumé de signalement vers une autre application.	Séance 3 - Intents	Bonus

9. Données manipulées et stockées

9.1 Principe

Les données à stocker sont définies à partir des besoins fonctionnels. Il n'existe pas de « liste officielle » à récupérer pour ce prototype : c'est l'équipe qui conçoit le modèle de données nécessaire au fonctionnement de TATITRA.

9.2 Dictionnaire de données principal

Donnée	Exemple	Local	Serveur	Sensibilité / remarque
ID local	8f71-...	Oui	Optionnel	Technique

Donnée	Exemple	Local	Serveur	Sensibilité / remarque
ID serveur	SIG-000125	Oui après sync	Oui	Technique
Catégorie	ROUTE	Oui	Oui	Non sensible
Description	Nid-de-poule important	Oui	Oui	Peut contenir texte libre
Latitude	-18.8792	Oui	Oui	Donnée de localisation
Longitude	47.5079	Oui	Oui	Donnée de localisation
Photo locale	/photos/a.jpg	Oui	Non	À supprimer selon politique
URL photo	https://.../a.jpg	Oui après sync	Oui	Référence fichier
Fichier photo	JPG/WebP	Temporaire	Storage	Peut contenir personnes/plaque : prudence
Statut	PRIS_EN_CHARGE	Oui	Oui	Métier
Synchronisé	false	Oui	Non	Technique
Date création	2026-09-10 14:30	Oui	Oui	Technique
Utilisateur	USR-104	Optionnel	Oui	Minimiser les données personnelles
Téléphone/e-mail	...	Optionnel	Optionnel	À éviter si non nécessaire au MVP
Commentaire admin	Équipe informée	Non	Optionnel	Métier

9.3 Où sont stockées les données ?

Emplacement	Contenu	Technologie
Téléphone	Signalement non synchronisé, métadonnées, chemin photo, statut local, notifications synchronisées	Room + stockage interne/app
Base serveur	Signalements, statuts, propositions/confirmations de résolution, échéances J+7, utilisateurs éventuels, historique	PostgreSQL/Supabase
Notifications	Destinataire, type d'événement, message, état lu/non lu, date	PostgreSQL/Supabase ; cache local possible
Stockage objet	Fichiers image	Supabase Storage
Navigateur admin	Pas de données métier permanentes nécessaires	Session/token uniquement si auth

10. Données de test et stratégie de démonstration

10.1 Deux types de données

Pour tester TATITRA, l'équipe peut combiner des données géographiques réelles et des incidents simulés. Une coordonnée réelle ne signifie pas qu'un problème existe réellement à cet endroit.

Type	Origine	Usage
Coordonnées GPS réelles	Téléphone ou carte publique	Tester latitude/longitude et affichage.
Routes / fonds cartographiques	OpenStreetMap / Geofabrik	Carte éventuelle et contexte géographique.
Limites administratives	ONU SALB / FTM	Bonus : région/district/commune.
Signalements	Créés par l'équipe	Tester le workflow métier.
Photos	Images de démonstration / galerie / placeholders	Tester upload et affichage.

10.2 Jeu de données simulées recommandé

ID	Catégorie	Description	Statut	isDemo
DEMO-001	ROUTE	Chaussée dégradée nécessitant une vérification	ENVOYE	true
DEMO-002	DECHETS	Accumulation de déchets sur un espace public	PRIS_EN_CHARGE	true
DEMO-003	ECLAIRAGE	Lampadaire public signalé comme hors service	A_VERIFIER	true
DEMO-004	DRAINAGE	Canal d'évacuation signalé comme bouché	RESOLU	true
DEMO-005	PONT	Dégradation à contrôler sur un ouvrage	EN_ATTENTE_SYNC	true

Règle académique importante

Les incidents de démonstration doivent être identifiés comme simulés. Ne placez pas un faux nid-de-poule à une adresse réelle en le présentant comme un fait. Un champ isDemo=true ou une mention « Données de démonstration » évite toute ambiguïté.

10.3 Tester depuis la maison

1. Précharger ou choisir une image de démonstration depuis la galerie.
2. Activer le mode avion.

3. Créer un signalement avec catégorie, description et photo.
4. Utiliser la position GPS du téléphone si disponible ; sinon utiliser une localisation de test clairement marquée.
5. Vérifier que le signalement apparaît en « En attente de synchronisation ».
6. Réactiver Internet.
7. Observer WorkManager envoyer le signalement.
8. Ouvrir l'interface Web admin sur le PC et vérifier la réception.
9. Changer le statut en « Pris en charge » puis vérifier la mise à jour côté mobile.
10. Depuis l'admin, proposer « Résolu » et vérifier que le citoyen voit « Résolution à confirmer ».
11. Pour tester la règle des 7 jours sans attendre une semaine, utiliser une date limite de test déjà expirée puis lancer le job de vérification ; le statut doit revenir à « Non résolu / endommagé ».

11. API REST

11.1 Endpoints MVP

Méthode	Endpoint	Rôle
POST	/api/signalements	Créer un signalement
GET	/api/signalements	Lister les signalements côté admin
GET	/api/signalements/:id	Afficher un détail
PATCH	/api/signalements/:id/statut	Modifier un statut de traitement
POST	/api/signalements/:id/resolution	Proposer une résolution (citoyen ou admin)
POST	/api/signalements/:id/resolution/confirm	Confirmer la résolution
POST	/api/signalements/:id/resolution/reopen	Indiquer toujours endommagé / rouvrir
GET	/api/notifications	Lister les notifications de l'utilisateur/admin
POST	/api/uploads	Uploader une photo, si séparé
GET	/health	Vérifier le backend

11.2 Exemple de requête

```
POST /api/signalements
Content-Type: application/json

{
  "clientId": "8f71-...",
  "categorie": "ROUTE",
  "description": "Chaussée dégradée - donnée de démonstration",
  "latitude": -18.8792,
  "longitude": 47.5079,
  "photoUrl": "https://.../demo01.jpg",
  "isDemo": true
}
```

11.3 Réponse

```
{
  "id": "SIG-000125",
  "statut": "ENVOYE",
  "createdAt": "2026-09-10T14:30:00+03:00"
}
```

11.4 Validation serveur

- Refuser une catégorie inconnue.
- Limiter la taille de la description.
- Valider que latitude est entre -90 et 90 et longitude entre -180 et 180.
- Limiter le type et la taille des images.
- Ne jamais faire confiance au rôle envoyé par le client.
- Prévoir une gestion d'erreurs claire : 400, 401/403, 404, 409, 500.

12. Interface Web d'administration

12.1 Choix retenu

Le projet ne nécessite pas une seconde application mobile pour l'administration. Une interface Web est plus rapide à développer, plus adaptée à un poste de travail administratif et partage le même backend que l'application Android.

12.2 Écran tableau de bord

ADMINISTRATION TATITRA

[Tous] [Envoyés] [Pris en charge] [A confirmer] [Rouverts] [Résolus]

#125	ROUTE	Analamanga	ENVOYE
#124	DECHETS	Analamanga	PRIS_EN_CHARGE
#123	ECLAIRAGE	Analamanga	RESOLU

[Voir détail]

12.3 Écran détail

SIGNALEMENT #125

[Photo]

Catégorie	ROUTE
Description	...
Latitude	...
Longitude	...
Date	...

Statut [PRIS_EN_CHARGE v]

[SIGNALER COMME RESOLU]

Si le citoyen a proposé la résolution :
[CONFIRMER RESOLU] [REFUSER / TOUJOURS EN COURS]

Echéance de confirmation : J+7
[ENREGISTRER]

12.4 Ce qui n'est pas nécessaire en 7 jours

- Gestion complexe des ministères/communes.
- Workflow multi-agent.
- Gestion des équipes de terrain.
- Système de permissions très granulaire.
- Dashboard BI avancé.

13. Sécurité, confidentialité et permissions

13.1 Permissions Android

Permission / capacité	Utilité	Comportement attendu
CAMERA	Prendre une photo	Demande au moment de l'usage.
ACCESS_FINE_LOCATION	GPS précis	Expliquer pourquoi la position est nécessaire.
ACCESS_COARSE_LOCATION	Position approximative	Fallback possible.
INTERNET	API et synchronisation	Pas de blocage de création si absent.
Galerie / Photo Picker	Image de démonstration	Préférer le sélecteur moderne sans permission large si possible.

13.2 Mesures

- HTTPS pour tous les échanges.
- Validation côté serveur.
- Rôles séparés citoyen/admin si authentification activée.
- Pas de stockage inutile de mots de passe en clair.
- URL signée ou règles d'accès adaptées pour les images si nécessaire.
- Minimisation des données personnelles.
- Éviter d'afficher publiquement l'identité du citoyen.
- Prévoir une mention sur les photos pouvant contenir des personnes ou plaques d'immatriculation.

14. Exigences non fonctionnelles

ID	Exigence	Cible MVP
NF-01	Utilisabilité	Créer un signalement en quelques étapes.
NF-02	Résilience réseau	Aucune perte lors d'une coupure Internet.
NF-03	Performance	Liste locale fluide sur un smartphone moyen.

ID	Exigence	Cible MVP
NF-04	Compatibilité	Android récent, cible adaptée au matériel disponible.
NF-05	Sécurité	HTTPS + validation serveur.
NF-06	Maintenabilité	Code séparé UI / data / réseau.
NF-07	Traçabilité	Logs utiles sans exposer de données sensibles.
NF-08	Accessibilité	Textes lisibles, actions identifiables, statut non basé seulement sur couleur.
NF-09	Cohérence métier	Toute proposition de résolution doit être traçable, confirmable et expirer après 7 jours sans confirmation.
NF-10	Automatisation	Le contrôle d'expiration J+7 s'exécute côté serveur, indépendamment de l'état du téléphone.
NF-11	Continuité de l'état	Une rotation/recréation d'Activity ne doit pas faire perdre un signalement ou l'état métier ; ViewModel + Room.
NF-12	Non-blocage UI	Aucun accès réseau, disque ou upload ne doit bloquer le thread principal ; utiliser coroutines / suspend.
NF-13	Réactivité	Les listes et statuts sont observés via Flow / StateFlow afin que Compose se recompose automatiquement.
NF-14	Préférences simples	Les réglages non relationnels (langue, onboarding, dernière synchro) utilisent DataStore si implémentés.

15. Modèle de données

15.1 Entités principales

```

UTILISATEUR
- id
- nom (optionnel)
- email/telephone (optionnels)
- role

SIGNALEMENT
- id
- client_id
- utilisateur_id (optionnel)
- categorie
- description
- latitude
- longitude
- photo_url
- statut
- resolution_proposee_par
- resolution_proposee_le
- date_limite_confirmation
- resolution_confirmee_le
- date_reouverture
- motif_reouverture

```

```
- is_demo
- date_creation
- date_modification
```

NOTIFICATION

```
- id
- destinataire_id
- signalement_id
- type
- message
- lue
- date_creation
```

TRAITEMENT (bonus)

```
- id
- signalement_id
- admin_id
- ancien_statut
- nouveau_statut
- commentaire
- date
```

15.2 États

```
EN_ATTENTE_SYNC -> ENVOYE -> A_VERIFIER -> PRIS_EN_CHARGE
|
+--> RESOLUTION_A_CONFIRMER
|       +--> RESOLU_CONFIRMER
|       +--> REOUVERT_NON_RESOLU
|       +--> expiration J+7 -> REOUVERT_NON_RESOLU
\--> REJETE
```

15.3 Exemple SQL

```
CREATE TABLE signalements (
  id UUID PRIMARY KEY,
  client_id UUID UNIQUE NOT NULL,
  categorie VARCHAR(40) NOT NULL,
  description TEXT NOT NULL,
  latitude NUMERIC(10,7),
  longitude NUMERIC(10,7),
  photo_url TEXT,
  statut VARCHAR(30) NOT NULL DEFAULT 'ENVOYE',
  resolution_proposee_par VARCHAR(20),
  resolution_proposee_le TIMESTAMPTZ,
  date_limite_confirmation TIMESTAMPTZ,
  resolution_confirmee_le TIMESTAMPTZ,
  date_reouverture TIMESTAMPTZ,
  motif_reouverture TEXT,
  is_demo BOOLEAN NOT NULL DEFAULT FALSE,
  date_creation TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  date_modification TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

16. Extraits de code proposés

16.1 Entité Room

```
@Entity(tableName = "signalements")
data class SignalementEntity(
    @PrimaryKey val idLocal: String,
    val categorie: String,
    val description: String,
    val latitude: Double?,
    val longitude: Double?,
    val photoPath: String?,
    val statut: String,
    val synchronise: Boolean = false,
    val serverId: String? = null,
    val dateCreation: Long
)
```

16.2 DAO

```
@Dao
interface SignalementDao {
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insert(item: SignalementEntity)

    @Query("SELECT * FROM signalements ORDER BY dateCreation DESC")
    fun observeAll(): Flow<List<SignalementEntity>>

    @Query("SELECT * FROM signalements WHERE synchronise = 0")
    suspend fun getPendingSync(): List<SignalementEntity>

    @Update
    suspend fun update(item: SignalementEntity)
}
```

16.3 Retrofit

```
interface TatitraApi {
    @POST("api/signalements")
    suspend fun createSignalement(
        @Body request: SignalementRequest
    ): SignalementResponse

    @GET("api/signalements/{id}")
    suspend fun getSignalement(
        @Path("id") id: String
    ): SignalementResponse
}
```

16.4 WorkManager : principe

```
val request = OneTimeWorkRequestBuilder<SyncSignalementsWorker>()
    .setConstraints(
        Constraints.Builder()
            .setRequiredNetworkType(NetworkType.CONNECTED)
```

```

        .build()
    )
    .build()

    WorkManager.getInstance(context).enqueue(request)

```

16.5 Backend Express : proposition de résolution

```

app.post('/api/signalements/:id/resolution', async (req, res) => {
  const { id } = req.params;
  const acteur = req.user.role; // CITOYEN ou ADMIN
  const deadline = new Date(Date.now() + 7 * 24 * 60 * 60 * 1000);

  await db.signalements.update(id, {
    statut: 'RESOLUTION_A_CONFIRMER',
    resolution_proposee_par: acteur,
    resolution_proposee_le: new Date(),
    date_limite_confirmation: deadline
  });

  // créer une notification pour l'autre partie
  res.json({ id, statut: 'RESOLUTION_A_CONFIRMER', deadline });
});

```

16.6 Backend : réouverture automatique après 7 jours

```

cron.schedule('0 * * * *', async () => {
  await db.query(`
    UPDATE signalements
    SET statut = 'REOUVERT_NON_RESOLU',
        date_reouverture = NOW(),
        motif_reouverture = 'Absence de confirmation sous 7 jours'
    WHERE statut = 'RESOLUTION_A_CONFIRMER'
      AND date_limite_confirmation <= NOW()
  `);
});

```

Pour la démonstration, il n'est pas nécessaire d'attendre réellement sept jours : une ligne de test peut recevoir une `date_limite_confirmation` passée, puis le job est lancé manuellement ou à fréquence courte dans l'environnement de test.

16.7 Repository - source locale d'abord

```

class SignalementRepository(
  private val dao: SignalementDao,
  private val api: TatitraApi
){
  fun observeAll(): Flow<List<SignalementEntity>> = dao.observeAll()

  suspend fun enregistrerLocal(item: SignalementEntity) {
    dao.insert(item.copy(synchronise = false))
  }

  suspend fun synchroniserEnAttente() {
    dao.getPendingSync().forEach { local ->

```

```

        val distant = api.createSignalement(local.toRequest())
        dao.update(local.copy(synchronise = true, serverId = distant.id))
    }
}

```

16.8 ViewModel + StateFlow

```

data class SignalementsUiState(
    val items: List<SignalementEntity> = emptyList(),
    val synchronisationEnCours: Boolean = false
)

class SignalementsViewModel(
    private val repository: SignalementRepository
) : ViewModel() {
    val uiState: StateFlow<SignalementsUiState> =
        repository.observeAll()
            .map { SignalementsUiState(items = it) }
            .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), SignalementsUiState())
}

```

16.9 Navigation Compose

```

NavHost(navController, startDestination = "accueil") {
    composable("accueil") { EcranAccueil(...) }
    composable("signalements") { EcranSignalements(...) }
    composable("signalement/{id}") { entry ->
        val id = entry.arguments?.getString("id")
        EcranDetail(id = id, onRetour = { navController.popBackStack() })
    }
}

```

16.10 DataStore - préférences simples

```

val LANGUAGE = stringPreferencesKey("langue")
val ONBOARDING_OK = booleanPreferencesKey("onboarding_ok")

val langue: Flow<String> = datastore.data
    .map { prefs -> prefs[LANGUAGE] ?: "fr" }

suspend fun enregistrerLangue(code: String) {
    datastore.edit { it[LANGUAGE] = code }
}

```

17. Tests et critères d'acceptation

17.1 Scénarios

Test	Étapes	Résultat attendu
T01 - Création online	Créer un signalement avec réseau.	Présent dans Room puis serveur ; synchronise=true.

Test	Étapes	Résultat attendu
T02 - Création offline	Mode avion puis création.	Présent dans Room ; EN_ATTENTE_SYNC.
T03 - Retour réseau	Réactiver Internet.	WorkManager envoie ; ID serveur mémorisé.
T04 - Persistance	Fermer complètement l'app puis rouvrir.	Signalement local toujours présent.
T05 - Photo	Ajouter une image.	Aperçu local puis URL serveur après upload.
T06 - GPS	Autoriser localisation.	Latitude/longitude enregistrées.
T07 - Refus GPS	Refuser permission.	Message clair ; comportement de secours.
T08 - Admin	Ouvrir détail et changer statut.	Nouvel état enregistré en base.
T09 - Serveur indisponible	Provoquer erreur API.	Pas de perte ; synchronise reste false.
T10 - Donnée de démo	Créer avec isDemo=true.	Identifiable comme démonstration.
T11 - Résolution par admin	Admin propose Résolu.	Citoyen voit RESOLUTION_A_CONFIRMER et une notification.
T12 - Résolution par citoyen	Citoyen propose Résolu.	Admin voit la proposition et peut confirmer/refuser.
T13 - Confirmation	L'autre partie confirme.	Statut devient RESOLU_CONFIRME.
T14 - Toujours endommagé	Citoyen clique Toujours endommagé.	Réouverture immédiate en REOUVERT_NON_RESOLU.
T15 - Expiration J+7	Date limite passée sans confirmation.	Job serveur remet automatiquement en REOUVERT_NON_RESOLU.
T08 - Rotation / recréation	Créer ou ouvrir un signalement puis tourner l'écran.	Aucune donnée métier perdue ; l'UI retrouve l'état via ViewModel / Room.
T09 - Navigation / retour	Liste → détail → retour système.	Retour correct à la liste, sans duplication d'écran.
T10 - Préférence DataStore	Changer la langue/préférence puis relancer l'app.	La préférence est relue au démarrage si DataStore est implémenté.

17.2 Définition de « terminé »

MVP accepté si...

Un signalement peut être créé en mode avion avec image + description + catégorie + localisation, persiste après redémarrage, se synchronise au retour du réseau, apparaît dans l'admin Web, peut être pris en charge, puis entrer en « Résolution à confirmer ». Le citoyen et l'admin peuvent chacun proposer/valider la résolution ; sans confirmation croisée à J+7, le dossier est automatiquement rouvert comme non résolu.

18. Organisation du binôme et planning 7 jours

18.1 Répartition recommandée

Membre A - Mobile	Membre B - Backend/Admin	En commun
Compose, Navigation Compose, MVVM/ViewModel/StateFlow, Repository, Room, DataStore, Camera/Galerie, GPS, Retrofit, Coroutines, WorkManager	Express, PostgreSQL/Supabase, Storage, endpoints, admin Web, job J+7	Modèle de données, Git, intégration, tests, Logcat, document, soutenance

18.2 Planning

Jour	Mobile	Backend/Admin	Commun
J1	Projet, Navigation Compose, modèles, squelette MVVM/Repository	Projet API, DB, schéma	Cahier des charges + maquettes
J2	Compose : formulaire + LazyColumn + StateFlow	POST/GET signalements	Contrats API
J3	Room + Repository + photo + GPS + DataStore	Upload image + validations	Tests unitaires simples
J4	Retrofit + coroutines + intégration API	Admin liste + détail	Intégration première version
J5	WorkManager + états sync + boutons résolution	API résolution + notifications + job J+7	Test offline/online
J6	Tests cycle de vie/navigation + corrections UI + confirmation	Admin confirmation/refus + réouverture	Scénario complet + test expiration + captures
J7	Finitions + build APK / démonstration	Déploiement démo	PDF, annexes, répétition soutenance

19. Risques et stratégie de réduction

Risque	Impact	Réponse
Trop de fonctionnalités	Élevé	Geler le MVP ; bonus seulement après validation du scénario principal.
Upload d'image instable	Élevé	Compresser, limiter taille, séparer métadonnées et fichier.
WorkManager non déclenché immédiatement	Moyen	Accepter l'exécution différée et proposer un bouton « Synchroniser ».
GPS indisponible	Moyen	Message clair et possibilité de réessayer.
Conflits Git	Moyen	Branches séparées mobile/backend et PR fréquentes.
Backend non déployé	Élevé	Tester en local/LAN puis déployer seulement lorsque stable.

Risque	Impact	Réponse
Fausse donnée présentée comme réelle	Élevé	Marquer isDemo=true et expliquer la méthodologie.
Job J+7 non exécuté	Moyen	Exécuter la logique côté serveur ; journaliser le job et ajouter un test avec échéance expirée.
Résolution fermée trop tôt	Moyen	Ne jamais passer directement à RESOLU_CONFIRME : imposer la confirmation croisée.

20. Déploiement et environnement

20.1 Développement

- VS Code + Claude Code comme environnement principal d'édition et de développement du code.
- Android SDK / Android Studio pour le SDK, l'émulateur, Logcat et les outils spécifiques Android.
- Gradle Wrapper du projet pour des builds reproductibles et indépendants du Gradle global de l'IDE.
- VS Code / IDE pour Node.js et admin Web.
- GitHub pour le code source.
- Postman/Bruno/curl pour tester l'API.
- Supabase ou PostgreSQL local/distant pour la base.
- node-cron (ou tâche planifiée équivalente) pour contrôler les échéances de résolution J+7.

20.2 Démonstration

Le backend peut être déployé sur un hébergement simple ou exécuté sur une machine accessible au téléphone. Le but de l'examen est de démontrer le fonctionnement ; une architecture nationale haute disponibilité n'est pas nécessaire au MVP.

20.3 Santé du backend

```
GET /health
{ "status": "ok" }
```

21. Fonctionnalités futures

- Carte nationale et clustering des incidents.
- Attribution automatique à la commune ou au service compétent.
- Notifications push système en complément des notifications in-app déjà prévues dans le MVP.
- Détection et fusion des signalements en doublon.
- Analyse d'image pour suggérer automatiquement la catégorie.
- Priorisation selon danger, fréquence ou proximité d'équipements sensibles.
- Multilingue Malagasy / Français.
- Gestion de plusieurs niveaux administratifs.
- Statistiques territoriales et exports.

- Cartes offline téléchargées à l'avance.
- Canal SMS/USSD pour certains usages sans smartphone.

22. Livrables

Livable	Contenu
Application Android	APK ou build + code source Kotlin.
Backend	API REST Node.js/Express.
Base	Schéma PostgreSQL/Supabase.
Admin Web	Liste, détail, traitement, proposition/confirmation de résolution et réouverture.
Repository Git	Historique du binôme.
Cahier des charges	Présent document.
Documentation technique	Architecture, API, données, sécurité.
Captures / Annexes	Écrans, tests offline/online, DB, admin.
Jeu de données de démonstration	Incidents simulés clairement marqués.
Kit branding TATITRA	logo_symbole.png, logos vertical/horizontal et icônes launcher, rangés dans les ressources Android et docs/branding.

23. Références et sources

Ces références servent à justifier le contexte et à fournir des sources de données géographiques possibles. Elles ne signifient pas que TATITRA est une application officielle existante.

Source	Utilité	Adresse
Ministère des Travaux Publics - Madagascar	Contexte : importance de l'entretien et de la modernisation du réseau routier.	https://mtp.gov.mg/mot-du-ministre
OpenStreetMap / Geofabrik - Madagascar	Données géographiques routières et fonds de carte.	https://download.geofabrik.de/africa/madagascar.html
ONU - SALB Madagascar	Limites administratives validées et formats GeoJSON/Shapefile/API.	https://salb.un.org/en/data/mdg
World Bank - Madagascar Roads	Jeu de données routier public historique pour référence/annexe.	https://datacatalog.worldbank.org/search/dataset/0039325/madagascar-roads
Documentation Android	Room, WorkManager, CameraX, localisation.	https://developer.android.com/
Supabase Documentation	PostgreSQL, Storage et services backend.	https://supabase.com/docs
ITU - État de l'art du développement mobile, M1 (2026)	Justification de Kotlin/Compose, architecture ViewModel/StateFlow/Repository/Room/Retrofit et offline-first.	Document pédagogique fourni
ITU - Syllabus M1	Traçabilité des compétences : Kotlin, coroutines, anatomie Android, Compose, navigation, MVVM et Room.	Document pédagogique fourni
ITU - Mini-TP 1 Kotlin essentiel	Null safety, data classes et collections Kotlin.	Document pédagogique fourni
ITU - Coroutines Kotlin	suspend, launch, async/await et non-blocage du thread UI.	Document pédagogique fourni
ITU - S3 Anatomie Android	Activity, cycle de vie, Intents et Logcat.	Document pédagogique fourni
ITU - S4 Jetpack Compose	Composables, état, recomposition et LazyColumn.	Document pédagogique fourni

Source	Utilité	Adresse
ITU - S5 Navigation	NavHost, routes, arguments et back stack.	Document pédagogique fourni
ITU - S6 MVVM	ViewModel, StateFlow, Repository et flux unidirectionnel.	Document pédagogique fourni
ITU - S7 Room et offline-first	Room/Flow, DataStore et projection Retrofit + WorkManager pour une vraie application.	Document pédagogique fourni

24. Annexes

Annexe A - Exemple de données JSON de démonstration

```
[
  {
    "clientId": "demo-001",
    "categorie": "ROUTE",
    "description": "Chaussée dégradée - donnée simulée",
    "latitude": -18.8792,
    "longitude": 47.5079,
    "statut": "ENVOYE",
    "isDemo": true
  },
  {
    "clientId": "demo-002",
    "categorie": "DECHETS",
    "description": "Accumulation de déchets - donnée simulée",
    "latitude": -18.88,
    "longitude": 47.51,
    "statut": "PRIS_EN_CHARGE",
    "isDemo": true
  }
]
```

Annexe B - Checklist de maintenance

- Le téléphone peut créer un signalement sans Internet.
- La photo de démonstration s'affiche.
- Les coordonnées sont visibles.
- Le signalement persiste après fermeture de l'application.
- Le statut « En attente de synchronisation » est clair.
- Au retour du réseau, la donnée arrive au backend.
- L'admin Web affiche le signalement.
- L'admin change le statut.
- L'admin peut proposer « Résolu » et le citoyen reçoit la demande de confirmation.
- Le citoyen peut aussi proposer « Résolu » et l'admin reçoit la demande.
- Le bouton « Toujours endommagé » rouvre immédiatement le dossier.
- Une échéance J+7 expirée rouvre automatiquement le dossier en test.
- Le document précise quelles données sont simulées.
- Les fonctionnalités non terminées sont annoncées comme perspectives et non comme réalisées.
- La rotation de l'écran ne fait pas perdre les données métier.
- La navigation liste → détail → retour respecte la back stack.
- Les appels réseau et base sont asynchrones et ne bloquent pas l'interface.

Annexe C - Arborescence de code suggérée

```

android-app/
data/
  local/      # Room : Entity, DAO, Database
  remote/     # Retrofit : API, DTO
  preferences/ # DataStore
  repository/ # arbitrage local / distant
domain/
ui/
  navigation/
  home/
  report/
  reports/
  components/
viewmodel/
worker/      # WorkManager
util/

backend/
src/
  routes/
  controllers/
  services/
  db/
  middleware/

admin-web/
src/
  pages/
  components/
  services/

docs/
branding/

```

Annexe D - Formulation à utiliser dans le rapport

Données de test

Les signalements utilisés pour la démonstration sont des données simulées conçues pour valider les fonctionnalités. Certaines coordonnées peuvent correspondre à des localisations géographiques réelles, mais elles ne constituent pas une affirmation qu'un incident existe actuellement à cet endroit.

Annexe E - Traçabilité des acquis M1 dans TATITRA

Séance / concept	Application concrète dans TATITRA	Preuve attendue
S1 - Kotlin essentiel	data classes, null safety, collections pour modèles et filtres.	Modèles Kotlin sans !! et code lisible.

Séance / concept	Application concrète dans TATITRA	Preuve attendue
S2 - Coroutines	Appels Room/Retrofit et synchronisation asynchrones.	Aucun blocage UI ; fonctions suspend.
S3 - Anatomie Android	Cycle de vie, Logcat, Intent de partage optionnel.	Rotation testée et logs filtrables.
S4 - Compose	UI déclarative, états visuels, LazyColumn.	Écrans réactifs sans findViewById.
S5 - Navigation	NavHost, détail par identifiant, retour back stack.	Liste → détail → retour fonctionnel.
S6 - MVVM	ViewModel + StateFlow + Repository.	Flux état descendant / événements remontants.
S7 - Room/offline-first	Room source locale, Flow, DataStore, WorkManager + Retrofit.	Création offline puis synchronisation.

Annexe F - Identité visuelle et fichiers de branding



Fichier	Rôle	Emplacements prévus
logo_symbole.png	Symbole seul (pin + point jaune)	drawable/ ; docs/branding/ ; admin-web/public/
logo_tatitra_vertical.png	Logo vertical (icône + texte)	drawable/ ; docs/branding/ ; admin-web/public/
logo_tatitra_horizontal.png	Logo horizontal	drawable/ ; docs/branding/ ; admin-web/public/
ic_launcher.png	Icône app - fond bleu uni	mipmap-* / ; docs/branding/
ic_launcher_round.png	Icône app - variante ronde / dégradée	mipmap-* / ; docs/branding/

FIN DU CAHIER DES CHARGES